

COE67-222 Operating Systems - Lab 05 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- memory_allocation.py before/after RAM usage: RAM เพิ่มจาก 13.81 MB เป็น 90.11 MB หรือเพิ่มขึ้นประมาณ 76.30 MB
- page_table_sim.py results: Logical Address 250 -> Physical Address 12250; Logical Address 3450 -> Physical Address 102450; Logical Address 9999 -> Page Fault
- ai_mmap_model.py RAM usage and accessed value: RAM เพิ่มจาก 13.96 MB เป็น 13.97 MB และค่าที่อ่านจาก index 25,000,000 คือ 0

1. Address Translation Math (Phase 3):

Assume an OS has a PAGE_SIZE of 4096 bytes.

If an AI model requests data at Logical Address 10000, calculate:

- The Page Number: 2
- The Offset: 1808

(Show your calculation steps briefly).

Page Number = $10000 // 4096 = 2$

Offset = $10000 \% 4096 = 1808$

2. Physical Memory Fragmentation:

In our page_table_sim.py, Page 0 was stored in Frame 12, and Page 1 was in Frame 45.

Why does the OS store continuous logical data in scattered/fragmented physical frames instead of putting them right next to each other? What problem does this solve?

Answer:

เพราะ RAM ที่ว่างอาจอยู่คนละตำแหน่งกัน OS เลยไม่จำเป็นต้องหาพื้นที่ว่างก้อนใหญ่ที่ต่อกัน แต่เอาแต่ละ Page ไปเก็บไว้ใน Frame ที่ว่าง แล้วใช้ Page Table ช่วยหาตำแหน่งจริง วิธีนี้ช่วยให้โปรแกรมใช้หน่วยความจำก้อนใหญ่ได้ แม้ RAM ที่ว่างจะกระเจายเป็นส่วนเล็ก ๆ

3. The AI Context (mmap):

In Phase 4, we mapped a 50MB file to memory, but the physical RAM usage did not increase significantly. Explain how the OS Virtual Memory and Page Table achieve this, and what actually happens at the OS level when index 25,000,000 is accessed.

Answer:

ตอนใช้ mmap โปรแกรมจะมองเห็นไฟล์เหมือนอยู่ในหน่วยความจำ แต่ OS ยังไม่ต้องโหลดไฟล์ทั้ง 50 MB เข้า RAM ทันที พอโปรแกรมเรียกใช้ index 25,000,000 จะเกิด Page Fault แล้ว OS จะโหลดเฉพาะส่วนที่มีข้อมูลตำแหน่งนั้นเข้า RAM จากนั้นโปรแกรมจึงอ่านค่าได้ ซึ่งในผลทดลองอ่านได้ค่า 0

Additional Summary and Application:

จาก Lab นี้ฉันเข้าใจว่า Logical Address กับ Physical Address เป็นคนละตำแหน่งกัน โปรแกรมจะเห็นตำแหน่งแบบ Logical ส่วน OS จะใช้ Page Table ช่วยหาตำแหน่งจริงใน RAM การใช้ mmap ก็ช่วยให้เปิดใช้ไฟล์ที่ใหญ่กว่า RAM ได้ เพราะไม่ต้องโหลดข้อมูลทั้งหมดเข้ามาพร้อมกัน แต่โหลดเฉพาะส่วนที่กำลังใช้งาน

COE67-222 Operating Systems - Lab 06 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- FIFO page faults: 9
- LRU page faults: 7
- Batch fits in RAM: 1.0206 seconds
- OS is swapping to disk: 2.3258 seconds
- Measured slowdown: Thrashing made the system 2 TIMES slower!

1. FIFO vs LRU (Phase 2):

In the `page_replacement.py` script, why did the LRU algorithm perform better (have fewer page faults) than the FIFO algorithm for the given reference string? Explain how LRU decides which page to evict.

Answer:

จากผลการทดลอง FIFO เกิด Page Fault 9 ครั้ง ส่วน LRU เกิด 7 ครั้ง LRU จึงดีกว่า 2 ครั้ง เพราะมันดูประวัติการใช้งานล่าสุด เมื่อ RAM เต็ม LRU จะนำหน้าที่ไม่ได้ถูกใช้นานที่สุดออกไป ทำให้หน้าที่เพิ่งถูกใช้งานยังอยู่ใน RAM ต่อ

FIFO ดูแค่ว่าหน้าที่ไหนเข้ามาก่อน จึงอาจนำหน้าที่กำลังจะถูกใช้อีกครั้งออกไป แม้หน้านั้นจะยังมีประโยชน์กับโปรแกรมอยู่

2. Understanding Thrashing (Phase 3):

Based on your observation of `ai_thrashing.py`, define "Thrashing" in your own words. Why is the processing time significantly higher in Scenario 2?

Answer:

ฉันเข้าใจว่า Thrashing คือภาวะที่ระบบเสียเวลาไปกับการย้ายข้อมูลระหว่าง RAM กับ Disk มากกว่าการประมวลผลจริง Scenario 2 ต้องเปิด อ่าน และเขียนไฟล์ซ้ำหลายรอบ ขณะที่ Scenario 1 เข้าถึงข้อมูลใน RAM โดยตรง จึงทำให้ Scenario 2 ใช้เวลามากกว่า

3. AI Engineering Decision:

If you are training an AI model and your system begins to thrash because the RAM is full, lowering the `batch_size` in your Python code is a common solution. How does lowering the `batch_size` help the Operating System resolve the Thrashing problem?

Answer:

การลด `batch_size` ทำให้ข้อมูลและค่าชั่วคราวที่ต้องเก็บในแต่ละรอบมีขนาดเล็กลง RAM หรือ VRAM จึงมีโอกาสปอสำหรับโมเดลมากขึ้น ระบบไม่ต้องย้ายหน้าไปมาระหว่าง RAM กับ Disk บ่อย ๆ ทำให้ Thrashing ลดลง

Additional Summary and Application:

Virtual Memory ช่วยให้โปรแกรมทำงานต่อได้เมื่อ RAM ไม่พอ โดยย้ายบางหน้าไปไว้บน Disk แต่ Disk ช้ากว่า RAM มาก สำหรับงาน AI ฉันจะเริ่มจากลด `batch_size` ตรวจสอบการใช้ RAM/VRAM และใช้เทคนิคอย่าง Gradient Accumulation หรือ Gradient Checkpointing แทนการพึ่ง OS Swap

COE67-222 Operating Systems - Lab 07 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from the sample process set

- FCFS Average Waiting Time: 7.33 time units
- FCFS Average Turnaround Time: 12.33 time units
- Round Robin Average Waiting Time: 4.33 time units
- Round Robin Average Turnaround Time: 10.00 time units

1. The Convoy Effect (Phase 2):

In the FCFS output of `cpu_scheduler.py`, what is the "Convoy Effect"? How did the presence of Process P1 affect the waiting times of P2 and P3?

Answer:

ใน FCFS P1 ใช้ CPU นาน 10 หน่วยเวลา ทำให้ P2 ต้องรอจนถึงเวลา 10 และ P3 ต้องรอจนถึงเวลา 12 แม้ทั้งสองงานจะสั้นมาก อาการที่งานสั้นต้องต่อแถวยาวหลังงานใหญ่คือ Convoy Effect

2. Choosing the Time Quantum:

In Round Robin, the Time Quantum is critical.

- What happens if the OS sets the Time Quantum extremely large (e.g., Quantum = 100)? Which algorithm does RR start to behave like?
- What happens if the OS sets the Time Quantum extremely small (e.g., Quantum = 0.0001)? (Hint: Think about Context Switching overhead.)

Answer:

ถ้า Quantum ใหญ่มาก งานแรกจะได้ใช้ CPU จนเกือบจบก่อนส่งต่อ จึงมีพฤติกรรมใกล้เคียง FCFS ถ้า Quantum เล็กมาก ระบบจะสลับงานถี่เกินไป ทำให้เสียเวลาไปกับ Context Switching มากจนการประมวลผลจริงช้าลง

3. AI Context:

In Phase 3, we used a Round Robin approach for generating tokens in an LLM. Explain why iteration-level Round Robin scheduling prevents long user queries from starving short queries, and how it improves perceived latency (Time To First Token / TTFT) for interactive AI applications.

Answer:

ระบบแบ่งการสร้าง Token ของแต่ละคำขอเป็นช่วงสั้น ๆ แล้วนำงานที่ยังไม่เสร็จกลับเข้าคิว User B จึงไม่ต้องรอให้คำตอบยาวของ User A สร้างครบก่อน ทำให้ได้ Token แรกเร็วขึ้นและรู้สึกระบบตอบสนองดีขึ้น ค่า TTFT จึงลดลง

Additional Summary and Application:

FCFS เข้าใจง่าย แต่ถ้างานแรกใหญ่ งานเล็กที่มาทีหลังจะรอนาน Round Robin แบ่งเวลาให้ทุกงาน จึงเหมาะกับ AI Inference API ที่มีผู้ใช้พร้อมกันจำนวนมากและลดการถูกงานยาวงานเดียวขวางคิวทั้งหมด

COE67-222 Operating Systems - Lab 08 Report

Name: Taofix Yayueri
Student ID: 67104000

Observed output from the sample disk requests

- FCFS Path: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
- FCFS Total Head Movement: 640 cylinders
- SCAN Path: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
- SCAN Total Head Movement: 331 cylinders
- Random small-file time: 0.0271 seconds
- Sequential packed-file time: 0.0009 seconds

1. Disk Scheduling (Phase 2):

Looking at the output of disk_scheduler.py, why did the SCAN algorithm result in a significantly lower "Total Head Movement" compared to FCFS? Explain the logic behind the Elevator behavior.

Answer:

FCFS ทำตามลำดับคำขอที่เข้ามา จึงทำให้หัวอ่านกระโดดไปมาระหว่างตำแหน่งไกล ๆ และเคลื่อนที่รวม 640 cylinders ส่วน SCAN เรียงคำขอแล้วเคลื่อนที่ไปทางเดียวจนถึงปลาย Disk ก่อนย้อนกลับ จึงลดระยะทางรวมเหลือ 331 cylinders

2. I/O Bound Concept:

If an Operating System is described as being "I/O Bound" during a task, what does that actually mean for the CPU? Is the CPU at 100% usage or is it mostly idle? Why?

Answer:

I/O Bound หมายถึงงานต้องรอการอ่านหรือเขียนข้อมูลเป็นหลัก CPU จึงไม่ได้ทำงานเต็ม 100% แต่มีช่วงว่างรอ Disk หรืออุปกรณ์ I/O เมื่อข้อมูลมาถึงจึงกลับมาประมวลผลต่อ

3. AI Industry Connection (Phase 3):

Explain why storing an AI image dataset as 1,000,000 separate .jpg files causes high I/O overhead and GPU starvation compared to packing them into a single sequential binary format (e.g., TFRecord, WebDataset, or HDF5).

Answer:

ไฟล์ .jpg จำนวนมากทำให้ OS ต้องเปิดไฟล์ ตรวจสอบ Metadata ตรวจสอบสิทธิ์ และปิดไฟล์ซ้ำเป็นล้านครั้ง จึงมี Overhead สูงและ GPU อาจว่างรอข้อมูล หากรวมเป็น TFRecord, WebDataset หรือ HDF5 ระบบจะอ่านข้อมูลต่อเนื่องได้ง่ายกว่าและส่งข้อมูลให้ GPU ได้สม่ำเสมอขึ้น

Additional Summary and Application:

Random Access มีข้อดีจากการกระโดดไปยังตำแหน่งต่าง ๆ และการจัดการไฟล์จำนวนมาก ส่วน Sequential Access อ่านข้อมูลต่อเนื่องจึงลดงานของ OS ได้มากกว่า ถ้า Data Pipeline เป็น I/O Bound GPU จะว่างงานแม้ตัว GPU จะยังมีความสามารถเหลืออยู่

COE67-222 Operating Systems - Lab 09 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- Inode Number and metadata: 1456337
- Default permissions: -rw-rw-rw-
- os_security.py: the write attempt should be blocked after 0o400
- ai_weight_protection.py: the write attempt should be blocked after 0o444

1. Inodes and Metadata:

In the inode_inspector.py script, we printed the "Inode Number". If you rename the file using the Linux mv command, will the Inode Number change? Why or why not? (Hint: Think about how the OS separates the file name from the actual data).

Answer:

ถ้าเปลี่ยนชื่อไฟล์ด้วย mv ภายใน File System เดียวกัน Inode Number จะไม่เปลี่ยน เพราะคำสั่งนี้เปลี่ยนชื่อที่ชี้ไปยังไฟล์ แต่ไม่ได้สร้างข้อมูลไฟล์ใหม่ รายละเอียดอย่างขนาด เวลา และสิทธิ์ยังอยู่กับ Inode เดิม

2. POSIX Permissions:

If you want to set a file's permission so that the Owner has Read/Write/Execute (7), the Group has Read/Execute (5), and Others have absolutely no access (0), what would be the 3-digit octal number you pass into os.chmod()?

Answer:

ค่าที่ใช้คือ 750 เพราะ Owner ได้ 7, Group ได้ 5 และ Others ได้ 0

3. The AI Context:

Why is it safer to rely on Operating System file permissions (like os.chmod(0o444)) to protect production model weights rather than relying solely on application-level logic or Python try-except blocks?

Answer:

Permission ของ OS ถูกตรวจตั้งแต่ตอนโปรเซสขอเปิดไฟล์และถูกบังคับใช้โดย Kernel ดังนั้นโปรแกรมที่พยายามเปิดไฟล์ในโหมดเขียนจะถูกปฏิเสธ แม้โค้ดแอปพลิเคชันจะเขียนเงื่อนไขป้องกันไว้ไม่ครบก็ตาม

Additional Summary and Application:

Inode เก็บ Metadata ของไฟล์ เช่น เจ้าของ ขนาด และตำแหน่งข้อมูล ส่วน Permission แบ่งสิทธิ์เป็น Owner, Group และ Others สำหรับ Model Weights ฉันจะใช้ os.chmod ให้เป็น Read-only และใช้ chown กำหนดเจ้าของหรือกลุ่มเมื่อมีสิทธิ์ วิธีนี้ป้องกันโปรเซสที่ไม่ควรแก้ไขไฟล์ได้ดีกว่าการตรวจสอบในโค้ดเพียงอย่างเดียว

COE67-222 Operating Systems - Lab 10 Report (Final)

Name: Taofix Yayueri

Student ID: 67104000

Observed output from the Live Dashboard

- Workload A: 40 GB RAM and GPUs 0, 1, 2
- Workload B: 16 GB RAM and CPU only
- Workload C: 2 GB RAM and GPU 3
- Workload D: rejected because secret_keys.txt was not found
- Dashboard screenshot: lab10/image.png

```
python3 ultimate_cluster_os.py
cd ..
=== Booting AI Cluster OS (64 GB RAM, 4 GPUs) ===
[API] Submitted: Workload_A_LLaMA -> Queued.
[WAIT] [Workload_A_LLaMA] Waiting for 40 GB RAM...
[MEMORY] [Workload_A_LLaMA] Allocated 40 GB RAM.
[WAIT] [Workload_A_LLaMA] Waiting for GPUs [0, 1, 2]...
[RUNNING] [Workload_A_LLaMA] Acquired GPUs [0, 1, 2].
[API] Submitted: Workload_B_Preproc -> Queued.
[WAIT] [Workload_B_Preproc] Waiting for 16 GB RAM...
[MEMORY] [Workload_B_Preproc] Allocated 16 GB RAM.
[RUNNING] [Workload_B_Preproc] Running on CPU only.

=====
[LIVE DASHBOARD] RAM Available: 8/64 GB
[LIVE DASHBOARD] GPU 0: BUSY (Workload_A_LLaMA) | GPU 1: BUSY (Workload_A_LLaMA) | GPU 2: BUSY (Workload_A_LLaMA) | GPU 3: IDLE
[LIVE DASHBOARD] Queue Size: 0 | Active Jobs: 2
=====

[API] Submitted: Workload_C_Infer -> Queued.
[WAIT] [Workload_C_Infer] Waiting for 2 GB RAM...
[MEMORY] [Workload_C_Infer] Allocated 2 GB RAM.
[WAIT] [Workload_C_Infer] Waiting for GPUs [3]...
[RUNNING] [Workload_C_Infer] Acquired GPUs [3].
[API] Submitted: Workload_D_Hacker -> Queued.
[FAIL] [Workload_D_Hacker] Dataset 'secret_keys.txt' not found on permission denied.

=====
[LIVE DASHBOARD] RAM Available: 6/64 GB
[LIVE DASHBOARD] GPU 0: BUSY (Workload_A_LLaMA) | GPU 1: BUSY (Workload_A_LLaMA) | GPU 2: BUSY (Workload_A_LLaMA) | GPU 3: BUSY (Workload_C_Infer)
[LIVE DASHBOARD] Queue Size: 0 | Active Jobs: 3
=====

=====
[LIVE DASHBOARD] RAM Available: 6/64 GB
[LIVE DASHBOARD] GPU 0: BUSY (Workload_A_LLaMA) | GPU 1: BUSY (Workload_A_LLaMA) | GPU 2: BUSY (Workload_A_LLaMA) | GPU 3: BUSY (Workload_C_Infer)
[LIVE DASHBOARD] Queue Size: 0 | Active Jobs: 3
=====

[DONE] [Workload_C_Infer] Finished successfully.

=====
```

Figure 1: Live Dashboard ขณะ Training, Preprocessing และ Inference ทำงานพร้อมกัน

1. The Role of the Operating System:

In this mini-project, our Python script simulated the major responsibilities of an Operating System.

Match the specific code behavior in ultimate_cluster_os.py to the corresponding OS concept you learned throughout the semester:

- File System & Permissions (Lab 9):

[Explain which line of code simulated this and why Workload D failed]

Answer:

โค้ดใช้ `os.path.exists(dataset_path)` ตรวจสอบไฟล์ก่อนเริ่มงาน Workload D ขอใช้ `secret_keys.txt` ซึ่งไม่มีอยู่ในระบบ จึงถูกปฏิเสธและไม่ได้รับทรัพยากรต่อ

- Memory Management (Labs 5/6):

[Explain how the OS prevented Thrashing by using the while loop for RAM allocation]

Answer:

while loop ตรวจสอบ available_ram_gb ก่อนหัก RAM ให้แต่ละงาน ถ้ายังมีไม่พอ งานจะรอแทนการคืนรัน วิธีนี้ช่วยไม่ให้ใช้ RAM เกินจริงจนเกิด Thrashing

- Concurrency & Deadlocks (Labs 3/4):

[Explain how the OS sorted the requested GPUs to avoid Circular Wait deadlocks]

Answer:

โค้ดใช้ sorted(req_gpus) ทำให้ทุกงานขอ GPU ตามลำดับเดียวกัน ลดโอกาสที่แต่ละงานจะถือ GPU บางตัวแล้วรอ GPU ของอีกงาน ซึ่งเป็นสาเหตุของ Circular Wait

2. Hardware Utilization:

Look at the output of your Live Dashboard. Explain how the OS successfully ran Workload A (LLaMA Training), Workload B (Preprocessing), and Workload C (Inference) AT THE SAME TIME without crashing. Why didn't they block each other?

Answer:

Workload A ใช้ GPU 0, 1, 2 และ RAM 40 GB ส่วน Workload B ไม่ใช้ GPU จึงทำงานบน CPU คบคู่กับ A ได้ ขณะที่ Workload C ใช้ GPU 3 ที่ยังว่างและ RAM 2 GB แต่ละงานใช้ทรัพยากรคนละส่วนหรืออยู่ภายใต้ Lock จึงไม่แย่งทรัพยากรกันจนระบบล่ม

3. Course Reflection (AI Engineering Context):

You are now applying for a role as a Machine Learning Operations (MLOps) Engineer.

Write a short paragraph explaining how understanding Operating Systems (Process queues, RAM/Virtual Memory limits, Deadlock avoidance, File systems) gives you an advantage over a programmer who only knows how to write basic Python/PyTorch scripts.

Answer:

ความรู้เรื่อง OS ทำให้ฉันมองเห็นปัญหาที่เกิดขึ้นบนระบบจริง เช่น งานติดที่ RAM, CPU Queue, Disk หรือสิทธิ์ไฟล์ จึงวางระบบ AI/MLOps ให้ใช้ทรัพยากรคุ้มค่า แก้ปัญหาได้เร็ว และป้องกันข้อมูลหรือ Model Weights เสียหายได้ดีกว่าการเขียนโค้ดให้ทำงานได้เพียงอย่างเดียว

Additional Capstone Summary:

Memory Allocation ป้องกันการใช้ RAM เกิน File System Security กรองไฟล์ที่เข้าถึงไม่ได้ การเรียงลำดับการขอ GPU ช่วยลด Deadlock และ Job Queue ช่วยจัดงานให้เหมาะกับทรัพยากร ทั้งสี่ส่วนทำให้ Training, Preprocessing และ Inference ทำงานพร้อมกันได้อย่างราบรื่น